

Seller and Inventory Management System

Scenario: An e-commerce enterprise handles merchandise sourced from a variety of suppliers. The database architecture must store supplier details, track merchandise provided by each supplier, and oversee stock levels to maintain supply readiness.

1. Create Seller and Inventory Tables:

- Form the Seller table structure with attributes for vendor identifier, vendor name, email address, contact phone, and location address.
- Form the Inventory table structure with attributes for stock identifier, item identifier, vendor identifier, stock level, availability condition, and update date.

```
SQL> CREATE TABLE Seller(Seller_ID VARCHAR(7), Seller_Name VARCHAR(25), Email VARCHAR(30), Phone_number NUMBER(10), Address VARCHAR(45));
Table created.

SQL> CREATE TABLE Inventory(Inventory_ID VARCHAR(7), Product_ID VARCHAR(7), Seller_ID VARCHAR(7), Stock_Quantity NUMBER, Stock_status VARCHAR(11), Last_Updated DATE);
Table created.
```

2. Establish Relationships:

Seller and Product:

- An individual supplier can provide multiple items.
- An individual item can be provided by single or multiple suppliers.

Relationship: Seller → Product operates as one-to-many.

Product and Inventory:

- Every item possesses corresponding stock records.
- Stock records track current item availability.

Relationship: Product → Inventory operates as one-to-one.

Primary Key added for Both Tables:

- Assign Primary Key constraint to the Seller table on the vendor identifier attribute.
- Assign Primary Key constraint to the Inventory table on the stock identifier attribute.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT PK_Seller PRIMARY KEY(Seller_ID);
Table altered.

SQL> ALTER TABLE Inventory ADD CONSTRAINT PK_Inv PRIMARY KEY(Inventory_ID);
Table altered.
```

Foreign Key for Inventory Table:

- Configure Foreign Key constraint on the Inventory table pointing to the item identifier in the merchandise table.
- Configure Foreign Key constraint on the Inventory table pointing to the vendor identifier in the seller table.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Prod FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID);
Table altered.

SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Seller FOREIGN KEY(Seller_ID) REFERENCES Seller(Seller_ID);
Table altered.
```

Constraints to make Unique Email ID:

- Enforce Unique constraint on the vendor email field.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT Un_mail UNIQUE(Email);
Table altered.
```

To make Email NOT NULL:

- Apply NOT NULL condition to the vendor email field.

```
SQL> ALTER TABLE Seller MODIFY Email NOT NULL;
Table altered.
```

Add Constraint to Check the Stock Quantity is Positive:

- Set Check constraint to ensure the stock level remains non-negative (>0).

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT CK_Quan CHECK(Stock_Quantity >= 0);
Table altered.
```

Insert Values to Seller Table:

Enter supplier entries interactively using substitution parameters.

```
SQL> INSERT INTO Seller VALUES('&SellID', '&SellerName', '&Email', &Phone, '&Address');
Enter value for sellid: S01
Enter value for sellername: Apex Global
Enter value for email: contact@apexglobal.com
Enter value for phone: 9812345678
Enter value for address: Bengaluru
old 1: INSERT INTO Seller VALUES('&SellID', '&SellerName', '&Email', &Phone, '&Address')
new 1: INSERT INTO Seller VALUES('S01', 'Apex Global', 'contact@apexglobal.com', 9812345678, 'Bengaluru')
1 row created.

SQL> /
Enter value for sellid: S02
Enter value for sellername: NextGen Logistics
Enter value for email: sales@nextgen.com
Enter value for phone: 9876512345
Enter value for address: Kolkata
old 1: INSERT INTO Seller VALUES('&SellID', '&SellerName', '&Email', &Phone, '&Address')
new 1: INSERT INTO Seller VALUES('S02', 'NextGen Logistics', 'sales@nextgen.com', 9876512345, 'Kolkata')
1 row created.

SQL> /
Enter value for sellid: S03
Enter value for sellername: Sunrise Traders
Enter value for email: info@sunrisetrade.com
Enter value for phone: 8765123490
Enter value for address: Hyderabad
old 1: INSERT INTO Seller VALUES('&SellID', '&SellerName', '&Email', &Phone, '&Address')
new 1: INSERT INTO Seller VALUES('S03', 'Sunrise Traders', 'info@sunrisetrade.com', 8765123490, 'Hyderabad')
1 row created.

SQL> /
Enter value for sellid: S04
Enter value for sellername: Prime Suppliers
Enter value for email: support@primesupply.com
Enter value for phone: 7891234560
Enter value for address: Pune
old 1: INSERT INTO Seller VALUES('&SellID', '&SellerName', '&Email', &Phone, '&Address')
new 1: INSERT INTO Seller VALUES('S04', 'Prime Suppliers', 'support@primesupply.com', 7891234560, 'Pune')
```

```
SQL> SELECT * FROM Seller;
```

SELLER_ID	SELLER_NAME	EMAIL	PHONE_NUMBER	ADDRESS
S01	Apex Global	contact@apexglobal.com	9812345678	Bengaluru
S02	NextGen Logistics	sales@nextgen.com	9876512345	Kolkata
S03	Sunrise Traders	info@sunrisetrade.com	8765123490	Hyderabad
S04	Prime Suppliers	support@primesupply.com	7891234560	Pune
S05	Urban Merchandise	urbanmerch@gmail.com	9012345678	New Delhi

Insert Values to Inventory Table:

Enter stock allocation entries linking suppliers, items, quantities, statuses, and dates.

```
SQL> INSERT INTO Inventory VALUES('I01', '101', 'S01', 20, 'Available', TO_DATE('2026-09-12', 'YYYY-MM-DD'));
1 row created.
```

```
SQL> INSERT INTO Inventory VALUES('I02', '102', 'S01', 35, 'Available', TO_DATE('2026-08-13', 'YYYY-MM-DD'));
1 row created.
```

```
SQL> INSERT INTO Inventory VALUES('I03', '103', 'S01', 50, 'Available', TO_DATE('2026-09-15', 'YYYY-MM-DD'));
1 row created.
```

```
SQL> INSERT INTO Inventory VALUES('I04', '104', 'S01', 15, 'Available', TO_DATE('2026-09-14', 'YYYY-MM-DD'));
1 row created.
```

```
SQL> INSERT INTO Inventory VALUES('I05', '105', 'S01', 30, 'Available', TO_DATE('2026-09-16', 'YYYY-MM-DD'));
1 row created.
```

```
SQL> SELECT * FROM Inventory;
```

INVENTORY_ID	PRODUCT_ID	SELLER_ID	STOCK_QUANTITY	STOCK_STATUS	LAST_UPDATED
I01	101	S01	20	Available	12-SEP-26
I02	102	S01	35	Available	13-AUG-26
I03	103	S01	50	Available	15-SEP-26
I04	104	S01	15	Available	14-SEP-26
I05	105	S01	30	Available	16-SEP-26

1. Seller-wise Product Report

Display:

- Vendor Name
- Quantity of items provided
- Total active stock

Use Joins to get details from Seller and Inventory Table.

```
SQL> SELECT s.Seller_Name, COUNT(i.Product_ID) AS Number_of_Products, SUM(i.Stock_Quantity) AS Available_Stock FROM Seller s JOIN Inventory i ON s.Seller_ID = i.Seller_ID WHERE i.Stock_Status = 'Available' GROUP BY s.Seller_Name;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
Apex Global	5	150

2. Stock Availability Report

Display:

- Item Name
- Stock Quantity
- Availability Status

```
SQL> SELECT p.Product_Name, i.Stock_Quantity, i.Stock_Status FROM Product p JOIN Inventory i ON p.Product_ID = i.Product_ID;
no rows selected
```

3: Inventory Analysis

Find:

- Total count of available items

```
SQL> SELECT COUNT(*) AS Total_Available_Products FROM Inventory WHERE Stock_Status = 'Available';
```

TOTAL_AVAILABLE_PRODUCTS
5

- Total count of out-of-stock items

```
SQL> SELECT COUNT(*) AS OutOfStock_Products FROM Inventory WHERE Stock_Status = 'Out of Stock';
```

OUTOFSTOCK_PRODUCTS
0

- Items holding peak stock quantities

```
SQL> SELECT p.Product_Name, i.Stock_Quantity FROM Product p JOIN Inventory i ON p.Product_ID = i.Product_ID WHERE i.Stock_Quantity = (SELECT MAX(Stock_Quantity) FROM Inventory);
```

no rows selected

- Mean stock level across all items

```
SQL> SELECT AVG(Stock_Quantity) AS Average_Inventory FROM Inventory;
```

AVERAGE_INVENTORY
30